Admin Panel To-Do Tracker

Phase 1 — Enhanced Ticket Management (Current)

- Implement Admin Notes Panel Visibility
  - Design UI for always-visible admin notes panel on ticket detail.
  - Develop front-end component to display admin notes panel on ticket detail page.
  - Integrate admin notes panel component into the existing ticket detail view.
- Implement Admin Notes Features
  - Design UI for displaying admin note author, timestamp, pin icon, edit, and delete buttons.
  - Develop back-end API endpoint for creating admin notes with author and timestamp.
  - Develop back-end API endpoint for pinning/unpinning admin notes.
  - Develop back-end API endpoint for editing existing admin notes.
  - Develop back-end API endpoint for deleting admin notes.
  - Develop front-end logic to display note details, handle pinning, and enable edit/delete actions.
- Implement Public/Private Ticket Notes
  - Design UI for a toggle switch or radio buttons to set note visibility (public/private).
  - Develop back-end logic to store note visibility status (public/private).
  - Develop back-end logic to enforce visibility rules based on user roles when retrieving notes.
  - Develop front-end logic to control the display of notes based on their visibility status and user role.
- Implement Ticket Tagging
  - Design UI for adding and removing tags directly on the ticket detail page.
  - Develop back-end API endpoint for adding tags to a specific ticket.
  - Develop back-end API endpoint for removing tags from a specific ticket.
  - Develop front-end logic to handle user input for adding new tags and clicking to remove existing tags.
- Implement Tag Management Page
  - Design UI for a dedicated tag management page including list, add, edit, delete functions, and a color picker.
  - Develop back-end API endpoints for creating new tags with name and color.
  - Develop back-end API endpoints for retrieving and listing all existing tags.
  - Develop back-end API endpoints for updating tag names and colors.
  - Develop back-end API endpoints for deleting tags.
  - Develop front-end components for displaying tag list, a form for adding/editing tags, and confirming deletions.
  - Implement client-side color picker functionality for tag creation/editing.
- Implement Inline Priority Change with Audit Trail
  - Design UI for an inline dropdown or selector for changing ticket priority and a mandatory reason input.
  - Develop back-end API endpoint for updating ticket priority.
  - Develop back-end service logic to record priority changes, including the old/new priority, author, timestamp, and reason, in an audit log.
  - Develop front-end logic to display the priority selector, capture the reason, and trigger the update.
  - Develop front-end component to display the audit trail entries related to priority changes on the ticket detail page.
- Implement Easy Navigation
  - Design UI for "Next Ticket" and "Previous Ticket" buttons on the ticket detail page.
  - Develop front-end logic to fetch and display the next/previous ticket based on the current list/filter context.
  - Implement global keyboard shortcut listeners for 'J' (previous) and 'K' (next) to navigate tickets.
- Implement Bulk Select in Tickets List
  - Design UI for checkboxes next to each ticket in the ticket list for bulk selection.
  - Develop front-end logic to enable single ticket selection and a "Select All" checkbox.
  - Implement front-end state management for tracking selected tickets.
- Implement Bulk Transfer
  - Design UI for a "Transfer" action button that appears when tickets are bulk-selected, with a dropdown for agent/team.
  - Develop back-end API endpoint for bulk transferring multiple tickets to a specified agent or team.
  - Develop front-end logic to send selected ticket IDs and the target agent/team to the bulk transfer API.
- Implement Bulk Status/Priority/Tag Updates
  - Design UI for "Update Status," "Update Priority," and "Update Tags" action buttons when tickets are bulk-selected.
  - Develop back-end API endpoint for bulk updating the status of multiple tickets.
  - Develop back-end API endpoint for bulk updating the priority of multiple tickets.
  - Develop back-end API endpoint for bulk adding/removing tags on multiple tickets.
  - Develop front-end logic to display appropriate modals/forms for bulk updates and send data to respective APIs.
- Implement Assignment Rules Engine
  - Define and document the logic for round-robin ticket assignment.
  - Define and document the logic for load-based ticket assignment (e.g., based on agent's current open tickets).
  - Define and document the logic for department/skill-based ticket assignment.
  - Develop back-end service responsible for evaluating and applying these assignment rules when a new ticket arrives.
- Implement Assignment Rules Admin UI
  - Design UI for an admin page to list, enable/disable, reorder, and preview assignment rules.
  - Develop back-end API endpoints for CRUD operations on assignment rules (e.g., create new rule, update parameters).
  - Develop back-end API endpoint for reordering the execution sequence of assignment rules.
  - Develop back-end API endpoint for simulating a rule's application given a sample ticket (preview functionality).
  - Develop front-end components for displaying rules, toggling their status, dragging to reorder, and interacting with the preview.

Phase 2 — Advanced Search & Filtering of tickets

- Implement Search by Mobile Number
  - Develop back-end API endpoint to search tickets based on customer mobile number.
  - Integrate mobile number search input into the main search bar or advanced filter section.
- Implement Search by Customer Email
  - Develop back-end API endpoint to search tickets based on customer email address.
  - Integrate customer email search input into the main search bar or advanced filter section.
- Implement Search by Customer Name
  - Develop back-end API endpoint to search tickets based on customer name (full or partial).
  - Integrate customer name search input into the main search bar or advanced filter section.
- Implement Search by Ticket ID
  - Develop back-end API endpoint to search tickets directly by their unique Ticket ID.
  - Integrate Ticket ID search input into the main search bar for quick access.
- Implement Filter by Product/Model
  - Develop back-end API endpoint to filter tickets based on associated product and/or model.
  - Design UI for a dropdown filter allowing selection of products and models.
  - Implement front-end logic to apply the selected product/model filter to the ticket list.
- Implement Saved Filters
  - Design UI for creating, listing, updating, and deleting user-specific saved filters.
  - Develop back-end API endpoints for creating and storing user-defined filter configurations.
  - Develop back-end API endpoints for retrieving a user's saved filters.
  - Develop back-end API endpoints for updating the parameters of an existing saved filter.
  - Develop back-end API endpoints for deleting saved filters.
  - Develop front-end components for managing and applying saved filters.
- Implement Export Current Result Set
  - Develop back-end API endpoint to generate an Excel file (.xlsx) containing the currently filtered ticket data.
  - Develop back-end API endpoint to generate a CSV file containing the currently filtered ticket data.
  - Design UI for an "Export" button with options for Excel and CSV formats.
  - Implement front-end logic to trigger the selected export format and initiate file download.

Phase 3 — Live Chat Enhancements

- Implement Agent Presence
  - Define agent presence states (online, away, busy, offline) and their transitions.
  - Develop back-end WebSocket service to manage and broadcast real-time agent presence updates.
  - Design UI for displaying an agent's current presence status (e.g., colored dot, status text).
  - Implement front-end logic for agents to manually change their presence status and for the UI to reflect it.
- Implement Capacity Limits Per Agent
  - Develop back-end logic to configure a maximum number of concurrent chats an agent can handle.
  - Implement back-end logic within the chat routing system to prevent assigning new chats to agents at full capacity.
  - Design UI in the admin panel for setting and modifying individual agent capacity limits.
- Implement Queue Overview
  - Develop back-end API endpoint to provide real-time data on chat queues, segmented by department.
  - Design UI for a dashboard displaying department-wise chat queue length and waiting times.
  - Implement front-end components to dynamically display the chat queue overview.
- Implement Customer Wait Timer
  - Develop back-end logic to calculate the exact waiting time for a customer if no agent is immediately available.
  - Implement front-end UI on the customer-facing chat widget to display the calculated waiting time.
- Implement Wait Time Estimate Heuristic (v1)
  - Develop an initial heuristic algorithm for estimating customer wait time (e.g., based on average handle time and queue length).
  - Implement back-end service to compute and provide the v1 wait time estimate.
  - Integrate the v1 wait time estimate into the customer chat widget.
- Implement Concurrency Alert
  - Develop back-end WebSocket service to track which agents are currently viewing a specific ticket or chat.
  - Design UI for a discreet alert or indicator when multiple agents are simultaneously viewing the same ticket/chat.
  - Implement front-end logic to display the concurrency alert to agents.
- Implement AI: Predicted Wait Time (v2)
  - Research and select a suitable machine learning model for more accurate wait time prediction.
  - Develop data pipelines for collecting and preparing historical chat data for model training.
  - Train and deploy the AI model for predicted wait time (v2) in a production environment.
  - Integrate the AI-predicted wait time (v2) into the chat routing and customer-facing interfaces.
- Implement AI: Auto-tagging and Suggested Replies (v2)
  - Research and select suitable machine learning models for auto-tagging and generating suggested replies.
  - Develop data pipelines for collecting and preparing chat transcripts for model training.
  - Train and deploy the AI models for auto-tagging and suggested replies (v2).
  - Integrate AI auto-tagging suggestions into the ticket creation and chat interfaces for agents.
  - Integrate AI suggested replies into the agent's chat input interface.

Phase 4 — Analytics & Reporting

- Implement Product-wise Analytics
  - Define and implement data aggregation logic for issues and resolution times per product.
  - Develop back-end API endpoint to retrieve product-wise issue counts and average resolution times.
  - Design UI for a dashboard section displaying product-specific analytics.
  - Develop front-end components to visualize product-wise issues and resolution metrics.
- Implement Issue-wise Analytics
  - Define and implement data aggregation logic for various issue types across tickets.
  - Develop back-end API endpoint to retrieve issue-wise counts, trends, and other relevant metrics.
  - Design UI for a dashboard section dedicated to issue-type analytics.
  - Develop front-end components to visualize common issues and their impact.
- Implement Ticket-wise TAT Exceeded Report
  - Define the criteria and business rules for "Time-to-Action (TAT) exceeded" tickets.
  - Develop back-end API endpoint to identify and generate a report of all tickets where TAT was exceeded.
  - Design UI for the TAT exceeded report, including filtering and sorting options.
  - Develop front-end components to display the TAT exceeded report.
- Implement Customer Feedback/CSAT Report
  - Integrate with the chosen CSAT survey system to collect and process customer feedback.
  - Develop back-end API endpoint to aggregate CSAT scores and display feedback trends.
  - Design UI for a CSAT report dashboard, including score trends and verbatim feedback.
  - Develop front-end components to visualize CSAT data and customer comments.
- Implement Agent Performance Reporting
  - Define key agent performance metrics: resolution rate, First Response Time (FRT), and CSAT scores.
  - Develop back-end API endpoint to calculate individual agent resolution rates.
  - Develop back-end API endpoint to calculate individual agent First Response Times (FRT).
  - Develop back-end API endpoint to aggregate individual agent CSAT scores.
  - Design UI for an agent performance dashboard displaying these key metrics.
  - Develop front-end components to display and compare agent performance data.
- Implement Department Analytics Dashboards
  - Aggregate relevant performance metrics (tickets handled, resolution times, FRT, CSAT) at the department level.
  - Develop back-end API endpoint to retrieve department-level analytics data.
  - Design UI for department-specific analytics dashboards, allowing drill-down views.
  - Develop front-end components to visualize department performance and trends.

Phase 5 — Departments & Teams

- Implement Departments List & Details
  - Define the data model for departments (e.g., name, description, assigned agents).
  - Develop back-end API endpoints for CRUD operations on department records.
  - Design UI for listing all departments and viewing/editing individual department details.
  - Develop front-end components for managing the list and detail views of departments.
- Implement Team Assignment Workflow
  - Define the workflow for assigning agents to specific teams within departments.
  - Develop back-end API endpoints for assigning agents to teams and removing them.
  - Design UI elements within the department/agent management pages for team assignment.
  - Implement front-end logic to manage agent-to-team assignments.
- Implement Department SLAs and Working Hours/Holidays
  - Define the data model for department-specific Service Level Agreements (SLAs).
  - Define the data model for department working hours and holiday schedules.
  - Develop back-end API endpoints for managing department SLAs, working hours, and holiday calendars.
  - Design UI for configuring and viewing department-specific SLAs and schedules.
  - Implement front-end forms and displays for department SLA and schedule configuration.
- Implement Ticket Routing to Departments
  - Develop back-end logic to route newly created tickets to specific departments based on predefined rules or manual selection.
  - Integrate department selection into the ticket creation and editing forms.
  - Design UI for a dropdown or selector to manually assign a ticket to a department during creation/editing.

Phase 6 — Time Tracking & SLAs

- Implement Agent Worklog
  - Define data model for agent worklog entries (e.g., start time, end time, associated ticket, activity type).
  - Develop back-end service to automatically start/stop worklog entries based on agent interaction with tickets.
  - Develop back-end API endpoints for agents to manually add, edit, or delete worklog entries.
  - Design UI for displaying an agent's daily/weekly worklog and for manual entry/editing.
  - Implement front-end components for agent worklog management and visualization.
- Implement Agent TAT per Ticket Computation
  - Define the specific calculation logic for "Time-to-Action (TAT)" for agents on a per-ticket basis.
  - Develop back-end service to compute and store the TAT for each agent on each ticket they handle.
  - Integrate the computed agent TAT into agent performance reports and ticket detail views.
- Implement SLA Policies per Department
  - (Dependency: Phase 5
  - Develop back-end service to actively apply the defined department-specific SLA policies to incoming and active tickets.
  - Implement back-end logic to track ticket progress against its assigned SLA.
- Implement SLA Risk Indicator and Alerts
  - Develop back-end service to monitor tickets against their SLAs and calculate a "risk" level (e.g., green, yellow, red based on time elapsed).
  - Develop back-end service to generate real-time alerts when a ticket approaches or exceeds its SLA.
  - Design UI for a visual SLA risk indicator on individual tickets and in ticket lists.
  - Implement front-end components to display SLA risk indicators and trigger in-app alerts.
- Implement TAT Reports
  - Develop back-end API endpoint to generate comprehensive TAT reports with various filters (e.g., by agent, department, time range, ticket type).
  - Develop back-end API endpoint to allow exporting TAT reports in Excel and CSV formats.
  - Design UI for a TAT report dashboard, including filtering options and an export button.
  - Implement front-end components to display and interact with the TAT reports.

Phase 7 — Notifications & Collaboration

- Implement In-app Notifications
  - Define types of in-app notifications (e.g., new assignment, @mention, SLA risk warning).
  - Develop back-end service to generate and store in-app notifications for relevant users.
  - Develop back-end API endpoints for retrieving a user's unread notifications and marking them as read.
  - Design UI for an in-app notification center (e.g., a bell icon with a badge, a dropdown list).
  - Implement front-end components to display and manage in-app notifications.
- Implement Email Notifications for Key Events
  - Define the specific key events that should trigger email notifications (e.g., ticket assigned, ticket status changed).
  - Develop back-end service to send email notifications for these predefined events.
  - Create and configure customizable email templates for different notification types.
- Implement @Mentions in Internal Comments
  - Develop back-end logic to parse internal comments for "@" mentions and identify mentioned users.
  - Develop back-end service to trigger in-app and/or email notifications to mentioned users.
  - Design UI for an auto-suggest feature when typing "@" in the comment box, showing available users.
  - Implement front-end functionality for typing @mentions and displaying them correctly in comments.
- Implement Presence Avatars on Ticket View
  - (Dependency: Phase 3
  - Develop back-end API endpoint to retrieve the presence status of agents currently viewing a specific ticket.
  - Design UI for displaying small avatars or indicators of agents who are currently active on the same ticket.
  - Implement front-end logic to fetch and display presence avatars on the ticket detail view.

Phase 8 — Products & Models

- Implement Product/Model Master Data Management
  - Define the data model for products (e.g., name, description) and models (e.g., name, associated product, dimensions).
  - Develop back-end API endpoints for CRUD operations on product records.
  - Develop back-end API endpoints for CRUD operations on model records.
  - Design UI for dedicated admin pages to manage product and model master data.
  - Implement front-end components for listing, adding, editing, and deleting products and models.
- Implement Document Attachment to Models
  - Develop back-end API endpoint for uploading documents and associating them with specific product models.
  - Implement a secure file storage solution (e.g., cloud storage, local storage) for model documents.
  - Design UI for an interface on the model detail page to upload, view, and manage attached documents.
  - Implement front-end components for document attachment and display on model pages.
- Implement Model Dimension in Analytics
  - (Dependency: Phase 4
  - Update the data schema and aggregation logic in the back-end to include 'model' as a primary dimension for analytics.
  - Modify existing back-end analytics APIs to allow filtering, grouping, and reporting by product model.
  - Update front-end analytics dashboards to incorporate model selection and display model-specific insights.
- Implement Model Selection in Ticket Forms
  - Develop back-end API endpoint to retrieve a list of available products and their associated models.
  - Design UI for a dropdown or cascading selection field for products and models in ticket creation/editing forms.
  - Implement front-end logic to dynamically populate model options based on product selection in ticket forms.

Phase 9 — Advanced Features

- Implement Dropdown-based Ticket Creation (Guided)
  - Design UI for a multi-step, dropdown-driven ticket creation flow (e.g., Category -> Sub-category -> Issue Type).
  - Develop back-end logic to provide dynamic options for subsequent dropdowns based on previous selections.
  - Implement front-end logic to guide the user through the dropdown-based ticket creation process.
- Implement Ticket Templates for Common Issues
  - Define the data model for ticket templates (e.g., template name, default title, description, tags, priority).
  - Develop back-end API endpoints for CRUD operations on ticket templates.
  - Design UI for an admin page to create, edit, and manage ticket templates.
  - Implement front-end functionality to select and apply a ticket template during ticket creation or editing.
- Implement Escalation Rules
  - Define the data model for escalation rules (e.g., trigger conditions like time elapsed, priority; actions like reassign, notify).
  - Develop back-end service to continuously evaluate active tickets against defined escalation rules.
  - Develop back-end API endpoints for CRUD operations on escalation rules.
  - Design UI for an admin page to configure and manage escalation rules.
  - Implement front-end components for defining escalation rule conditions and actions.
- Implement External Integration APIs
  - Research and identify common external systems (e.g., CRM, ERP, payment gateways) for potential integration.
  - Design and document API specifications (e.g., REST endpoints, webhooks) for various integration points.
  - Develop a secure back-end framework to support and manage external API integrations (e.g., authentication, logging).
  - Create comprehensive API documentation for developers looking to integrate with the admin panel.

Presented with xmind